

Institutional System Health & Architecture Report

Document Status: Final System Blueprint (Asset Data Redacted for Privacy)
System Iteration: Sniper V5 (Forward-Testing Protocol)
Prepared For: CA Sunil Kumar Paswan, Quantitative Systems Architect (Jamadoba Operations)

1. Executive Summary & Historical Context

Following a deep quantitative audit of historical trading performance, it was identified that a significant drawdown of ₹5 Lakhs was not the result of algorithmic failure, but rather manual discretionary execution errors—specifically overtrading, emotional hijacking, cutting profits prematurely, and ignoring hard stop-losses. The core underlying algorithmic logic (Breakout velocity × Trend efficiency) possesses a robust mathematical edge.

The **Sniper V5 Architecture** documented herein serves as a strict procedural and technical safeguard. By transitioning the role from a discretionary trader to a Systems Architect, the system now enforces institutional-grade constraints, neutralizes psychological friction, and fully automates exit management via Telegram telemetry. Specific stock tickers have been obfuscated to protect the proprietary nature of this alpha.

2. Deep Analytics Validation (1-Year Quantitative System Audit)

A granular 1-year backtest involving precise F&O taxation, slippage (0.5%), and theta decay penalties revealed the exact behavioral profile of the selected assets. The data proves a strong **1:2.5 Risk/Reward ratio**, demonstrating that the system absorbs small, calculated paper cuts while capturing massive high-beta trends.

Asset Profile (Anonymized)	Win Rate	Average Win	Average Loss	Profit Factor	Optimal Time (IST)
Asset 1	34.68%	₹ 4,257.13	₹ -1,686.14	1.34	02:30 PM
Asset 2	33.52%	₹ 4,318.98	₹ -1,681.57	1.30	02:30 PM
Asset 3	35.99%	₹ 3,250.13	₹ -1,436.86	1.27	02:30 PM
Asset 4	34.13%	₹ 2,896.54	₹ -1,208.81	1.24	12:30 PM
Asset 5	33.01%	₹ 3,876.06	₹ -1,649.38	1.16	01:30 PM

3. Immutable System Rules & Constraints

Rule 1: Asset Pruning

Any asset with a Profit Factor below 1.20 fails to overcome real-world slippage and taxes efficiently. **Asset 5** has been permanently excised from the live universe.

Rule 2: Time Lockout (The 12:30 PM Rule)

The audit revealed that morning breakouts are largely institutional traps resulting in high-frequency stop-loss triggers. The live scanner is hard-coded to ignore all market data prior to **12:30 PM IST**, aligning exclusively with European open volatility and the late Indian institutional power hour.

Rule 3: Ammunition Limits

The system will forcibly power down its scanning engine upon triggering a maximum of **2 signals per day**. This is the ultimate defense against overtrading and revenge trading.

Rule 4: Autonomous Exit Management

Discretionary exits are strictly prohibited. The system actively monitors the live spot price and will issue a Telegram command to exit based on three conditions:

1. **Hard SL:** Spot crosses the breakout candle boundary.
2. **Trailing SL:** Spot pulls back 0.5% from the peak high/low.
3. **Theta Limit:** Position held for over 65 minutes with no momentum.

4. Final Architecture Code (Sniper V5)

Below is the complete, final deployment code to be executed via a dedicated Python environment. The system utilizes the `dhank` library for market data and the Telegram API for real-time telemetry. Stock tickers in the code block have been masked as "ASSET_1", "ASSET_2", etc., to preserve the secrecy of the strategy parameters.

```

import os
import time
import pandas as pd
import pandas_ta as ta
import numpy as np
import datetime
import requests
from dhanhq import dhanhq

# =====
# 1. INSTITUTIONAL SYSTEM CONFIGURATION
# =====
CLIENT_ID = "YOUR_CLIENT_ID_HERE"
ACCESS_TOKEN = "YOUR_NEW_SECURE_DHAN_TOKEN_HERE"

TELEGRAM_BOT_TOKEN = "YOUR_NEW_SECURE_TELEGRAM_BOT_TOKEN"
TELEGRAM_CHAT_ID = "YOUR_CHAT_ID"

# Pruned Universe: Only Profit Factors > 1.20 (Underperforming Asset 5 removed)
# TICKERS BLURRED/REDACTED FOR PROPRIETARY PROTECTION
TARGET_TICKERS = ["ASSET_1", "ASSET_2", "ASSET_3", "ASSET_4"]

# Ammunition & Time Limits
MAX_SIGNALS_PER_DAY = 2
TIME_LOCKOUT_HOUR = 12
TIME_LOCKOUT_MINUTE = 30
THETA_LIMIT_MINUTES = 65
TRAILING_SL_PCT = 0.005 # 0.5%

try:
    print("🔄 Initializing Systems & Connecting to Dhan API...")
    dhan = dhanhq(CLIENT_ID, ACCESS_TOKEN)
    print("✅ Connection Verified.")
except Exception as e:
    print(f"❌ Critical Auth Error: {e}")
    exit()

# =====
# 2. TELEGRAM COMMUNICATION ENGINE
# =====
def send_telegram_alert(message):
    """Fires formatted alerts to your Telegram device."""
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {"chat_id": TELEGRAM_CHAT_ID, "text": message, "parse_mode": "Markdown"}
    try:
        requests.post(url, json=payload)
    except Exception:
        pass

# =====
# 3. DATA ACQUISITION
# =====
def download_dhan_master():
    """Fetches the daily scrip master to get internal Security IDs."""
    url = "https://images.dhan.co/api-data/api-scrip-master.csv"
    try:
        df = pd.read_csv(url, low_memory=False)
        df.columns = [str(col).strip() for col in df.columns]
        return df[df['SEM_SERIES'].isin(['EQ', 'BE'])]
    except Exception:
        return None

```

```

def get_latest_5min_data(sec_id):
    """Fetches today's intraday data and formats it into 5min candles."""
    today = datetime.datetime.now().strftime("%Y-%m-%d")
    try:
        req = dhan.intraday_minute_data(
            security_id=str(sec_id), exchange_segment='NSE_EQ',
            instrument_type='EQUITY', from_date=today, to_date=today
        )
        if req.get('status') == 'success' and req.get('data'):
            df = pd.DataFrame(req['data'])
            if df.empty: return None

            sample = df['timestamp'].iloc[0]
            df['timestamp'] = pd.to_datetime(df['timestamp'], unit='ms' if sample > 1e11 else
's')

            # Adjust UTC to IST based on API return formatting
            df['timestamp'] = df['timestamp'] + datetime.timedelta(hours=5, minutes=30)
            df.set_index('timestamp', inplace=True)
            for col in ['open', 'high', 'low', 'close', 'volume']:
                df[col] = df[col].astype(float)
            df.sort_index(inplace=True)
            return df.resample('5min').agg({'open': 'first', 'high': 'max', 'low': 'min',
'close': 'last', 'volume': 'sum'}).dropna()
        except Exception:
            pass
    return None

# =====
# 4. MASTER LIFECYCLE CONTROLLER
# =====
def run_sniper_manager():
    eq_master = download_dhan_master()
    if eq_master is None:
        print("❌ Failed to load Scrip Master. Check API.")
        return

    init_msg = (f"🟢 *SNIPER V5 SYSTEM ONLINE*\n\n"
        f"🛡️ *Active Assets:* {len(TARGET_TICKERS)}\n"
        f"🕒 *Time Lockout:* Until {TIME_LOCKOUT_HOUR}:{TIME_LOCKOUT_MINUTE} PM IST\n"
        f"🎯 *Daily Limit:* {MAX_SIGNALS_PER_DAY} Trades\n"
        f"🤖 *Manager:* Trailing SL & Theta tracking engaged.")
    send_telegram_alert(init_msg)
    print("\n" + "="*60)
    print("🎯 SNIPER V5: INSTITUTIONAL FORWARD-TESTING MODE")
    print("="*60 + "\n")

    recent_signals = {}
    active_trades = {}
    signals_today = 0
    current_date = datetime.datetime.now().date()

    while True:
        now = datetime.datetime.now()

        # --- DAILY RESET PROTOCOL ---
        if now.date() > current_date:
            current_date = now.date()
            signals_today = 0
            recent_signals.clear()
            active_trades.clear()
            send_telegram_alert(f"☀️ *New Trading Day*\nAmmunition reset to
{MAX_SIGNALS_PER_DAY}.")

```

```

# --- MARKET HOURS PROTOCOL ---
is_market_open = (now.hour > 9 or (now.hour == 9 and now.minute >= 15)) and (now.hour <
15 or (now.hour == 15 and now.minute < 30))
if not is_market_open:
    print(f"[{now.strftime('%H:%M:%S')}] Market Closed. Standing by...", end="\r")
    time.sleep(60)
    continue

# =====
# PHASE 1: ACTIVE EXIT MANAGEMENT
# =====
for ticker in list(active_trades.keys()):
    trade = active_trades[ticker]
    df = get_latest_5min_data(trade['sec_id'])
    if df is None: continue

    current_spot = df['close'].iloc[-1]
    time_held_mins = (now - trade['entry_time']).total_seconds() / 60.0

    exit_signal = False
    exit_reason = ""

    if trade['opt_type'] == 'CE':
        # Update Trailing SL (lock in profits as it goes up)
        if current_spot > trade['peak_spot']:
            trade['peak_spot'] = current_spot
            trade['trailing_sl'] = max(trade['trailing_sl'], current_spot * (1 -
TRAILING_SL_PCT))

        # Check Exits
        if current_spot <= trade['hard_sl']:
            exit_signal, exit_reason = True, "🔴 HARD STOP LOSS HIT (Spot dropped below
candle low)"
        elif current_spot <= trade['trailing_sl']:
            exit_signal, exit_reason = True, "📉 TRAILING STOP HIT (0.5% Pullback from
Peak)"

    else: # PE
        # Update Trailing SL (lock in profits as it goes down)
        if current_spot < trade['peak_spot']:
            trade['peak_spot'] = current_spot
            trade['trailing_sl'] = min(trade['trailing_sl'], current_spot * (1 +
TRAILING_SL_PCT))

        # Check Exits
        if current_spot >= trade['hard_sl']:
            exit_signal, exit_reason = True, "🔴 HARD STOP LOSS HIT (Spot broke above
candle high)"
        elif current_spot >= trade['trailing_sl']:
            exit_signal, exit_reason = True, "📈 TRAILING STOP HIT (0.5% Pullback from
Peak)"

    # Check Theta Decay Limit
    if time_held_mins >= THETA_LIMIT_MINUTES and not exit_signal:
        exit_signal, exit_reason = True, f"⌚ TIME DECAY LIMIT ({THETA_LIMIT_MINUTES}
Min Chop)"

    # Execute Exit Alert
    if exit_signal:
        msg = (f"🔴 *EXIT COMMAND: {ticker}* 📢\n\n"
            f"🟡 *Reason: {exit_reason}\n"
            f"💰 *Exit Spot: ₹{current_spot:.2f}\n"
            f"🕒 *Time Held: {int(time_held_mins)} mins\n\n")

```

```

        f"🛒 *Action:* CLOSE {trade['opt_type']} POSITION IMMEDIATELY at Market
Price.")

    send_telegram_alert(msg)
    print(f"\n🔴 {msg.replace('*', '')}\n")
    del active_trades[ticker]

# =====
# PHASE 2: ENTRY SCOUTING
# =====
# Only scout if we haven't hit max signals AND it is past 12:30 PM IST (Time Lockout)
is_past_lockout = now.hour > TIME_LOCKOUT_HOUR or (now.hour == TIME_LOCKOUT_HOUR and
now.minute >= TIME_LOCKOUT_MINUTE)

if signals_today < MAX_SIGNALS_PER_DAY and is_past_lockout and now.minute % 5 == 1:
    print(f"[{now.strftime('%H:%M:%S')}] Scouting Power Hour Setups | Active Trades:
{len(active_trades)}...", end="\r")

for ticker in TARGET_TICKERS:
    if signals_today >= MAX_SIGNALS_PER_DAY or ticker in active_trades:
        continue

    eq_row = eq_master[eq_master['SEM_TRADING_SYMBOL'] == ticker]
    if eq_row.empty: continue

    sec_id = str(eq_row.iloc[0]['SEM_SMST_SECURITY_ID'])
    df = get_latest_5min_data(sec_id)

    if df is None or len(df) < 20: continue

    try:
        adx = df.ta.adx(length=14)
        df = pd.concat([df, adx], axis=1)
        df['atr'] = df.ta.atr(length=14)
        df['tr'] = df.ta.true_range()
        df['vol_sma'] = df['volume'].rolling(window=20).mean()
        df['ema_9'] = ta.ema(df['close'], length=9)
    except Exception:
        continue

    latest_candle = df.iloc[-2]
    candle_time = df.index[-2].strftime("%H:%M")

    # Institutional Breakout Logic
    if (latest_candle['ADX_14'] > 25 and
        latest_candle['volume'] > (latest_candle['vol_sma'] * 2.0) and
        latest_candle['tr'] > (latest_candle['atr'] * 1.5)):

        spot = latest_candle['close']
        opt_type = "CE" if spot > latest_candle['ema_9'] else "PE"
        sl_level = latest_candle['low'] if opt_type == 'CE' else
latest_candle['high']

        signal_key = f"{ticker}_{candle_time}"

        if signal_key not in recent_signals:
            signals_today += 1
            recent_signals[signal_key] = True

        # Register trade for automated exit management
        active_trades[ticker] = {
            'sec_id': sec_id,
            'opt_type': opt_type,
            'entry_time': now,

```

```

        'peak_spot': spot,
        'hard_sl': sl_level,
        'trailing_sl': spot * (1 - TRAILING_SL_PCT) if opt_type == 'CE'
    else spot * (1 + TRAILING_SL_PCT)
    }

    tg_message = (
        f"🎯 *SNIPER SIGNAL [{signals_today}/{MAX_SIGNALS_PER_DAY}] * 🎯

\n\n"

        f"📈 *Stock:* {ticker}\n"
        f"🛒 *Trade:* BUY {opt_type}\n"
        f"💰 *Spot Price:* ₹{spot:.2f}\n"
        f"🛡️ *Hard Stop Loss:* ₹{sl_level:.2f}\n\n"
        f"🤖 *Manager Status:* Logged. Awaiting exit conditions..."
    )
    send_telegram_alert(tg_message)
    print(f"\n🎯 [ENTRY] {ticker} {opt_type} @ {spot:.2f} logged into
Manager.")

    # Sleep 60s so we don't scan the exact same minute twice
    time.sleep(60)
else:
    # Display standby status based on conditions
    if signals_today >= MAX_SIGNALS_PER_DAY:
        status = "Max Daily Ammunition Used."
    elif not is_past_lockout:
        status = f"Time Lockout Active (Waiting for {TIME_LOCKOUT_HOUR}:
{TIME_LOCKOUT_MINUTE})."
    else:
        status = f"Waiting for next candle. Active: {len(active_trades)}"

    print(f"[{now.strftime('%H:%M:%S')}] {status}", end="\r")
    time.sleep(15)

if __name__ == "__main__":
    run_sniper_manager()

```

"Abdicate the role of the Trader. Assume the role of the Systems Architect."

Proprietary Strategy Documentation - Generated exclusively for CA Sunil Kumar Paswan.